

Project 1 Boids

Tommaso Ciccotti
7196985

tommaso.ciccotti@edu.unifi.it

Abstract

This project implements a boids flocking simulation with a sequential baseline and an OpenMP version. The update of each boid depends on a read-only snapshot of the current flock. This allows the update loop to be distributed across CPU threads without locks, critical sections or atomic operations. Four neighbour-search profiles are evaluated: Naive, Squared Distance, Bounding Box and Grid. The first three retain an all-pairs traversal while grid restricts the search to the current spatial cell and its neighbours. The benchmark covers four flock sizes, eight thread counts, two OpenMP scheduling policies and six chunk sizes. A separate sequential reference is recorded for every profile and population. Results show that useful scaling continues up to the 12 logical execution contexts of the processor while oversubscribed configurations provide only small and inconsistent changes. The Grid profile produces the largest absolute improvement because it reduces the number of candidate neighbours before parallel execution begins. Its advantage grows as the flock becomes larger. Dynamic scheduling is usually faster than static scheduling but the scheduling effect remains smaller than the difference among neighbour-search methods.

Future distribution permission

The author of this report give permission for this document to be distributed to Unifi affiliated students taking future courses.

1. Introduction

A boids simulation contains autonomous agents that update their positions at each frame from their own state and from nearby agents. In the original all-pairs method every boid checks the complete population. The work therefore grows quadratically with the number of agents and the sequential execution time rises rapidly.

The update can be parallelized because every

boid reads the same current flock state and writes only its own pending state. After all pending states have been calculated a second loop commits the new positions and velocities.

The motion follows three interaction rules: cohesion, separation and alignment. The project starts from a sequential implementation and introduces OpenMP parallelization. It also evaluates three arithmetic or spatial optimizations and SFML provides the real-time visualization.

The benchmark separates the effect of the neighbour-search method from the effects of the thread count and the scheduling.

The simulation also provides a useful parallel-programming case study. The mathematical rules are simple but the cost of finding relevant neighbours dominates the execution while a faster distance formula changes the cost of each comparison. A spatial index changes how many comparisons are required while OpenMP changes how the remaining work is shared among processor threads.

2. Implementation

2.1. Sequential path

In the sequential baseline each boid is treated as an independent agent represented by a two-dimensional shape moving within a 2D space. During the update cycle three primary rules control its behaviour: *cohesion* (steering toward the center of mass of its local neighbours), *separation* (steering to avoid crowding or colliding with close neighbours), and *alignment* (steering to match the general heading of the flock).

The original algorithm operates by scanning

every other boid in the system to calculate the pairwise Euclidean distances and compute the force vectors required by the three components. Furthermore, the sequential implementation limits the maximum velocity to enforce stability constraints and applies steering adjustments to keep the boids within the allowed window boundaries.

Because each agent must completely scan the entire population array the algorithm exhibits a strict quadratic complexity of $\mathcal{O}(N^2)$ per frame. The optimization and parallelization of the algorithm focus directly on reducing the complexity of the individual spatial queries and parallelizing the update routine for each boid.

2.2. OpenMP path

At frame $t + 1$ the state of the boid i depends on the read-only states of the flock at frame t :

$$\text{state}_i^{(t+1)} = f\left(\text{state}_i^{(t)}, \{\text{state}_j^{(t)} \mid j \neq i\}\right) \quad (1)$$

The function $f(\cdot)$ combines cohesion, separation and alignment. New positions and velocities are stored in pending fields.

The parallel version creates one OpenMP region with `#pragma omp parallel`. The update loop and the commit loop use `#pragma omp for`. Each iteration writes only the pending state of its assigned boid and the accumulators are local to it. The barrier after the update loop ensures the completion of every pending state before the commit phase starts. No locks, critical sections or atomic operations are required.

The loops use `schedule(runtime)`. The benchmark combines static and dynamic scheduling with different chunk sizes.

2.3. Correctness of the update

During the first loop every thread reads positions and velocities from the current state and stores the results in pending fields belonging to the same boid, no thread changes the current position while another thread is using it. During the second loop the new positions and velocities are finalized

The implicit barrier at the end of the update loop is required. Without it a thread could commit a new state while another thread was still reading the old flock. The second loop starts only after all pending states have been computed.

2.4. Work distribution

The first loop is a naturally suited for parallelization because one iteration contains the complete update of one boid. The amount of work per iteration depends on the selected profile. Naive, Squared Distance and Bounding Box inspect the full population, their iteration costs are similar within the same frame.

Grid iterations differ more, a boid in a dense region inspects more entries than a boid in a sparse region. Dynamic scheduling can redistribute these uneven iterations while static scheduling has lower assignment overhead.

The commit loop has little work compared to the neighbour search. It still runs inside the same OpenMP region so a new team is not created for the second phase. This avoids one additional parallel-region startup per frame.

3. Optimizations

Various optimization profiles have been used and each of them has different performances.

3.1. Naive

The Naive approach serves as the baseline for evaluating performance scaling. This profile treats the flock as a fully connected graph where every agent must interact with every other. In a 2D continuous space the Euclidean distance (d) between a boid i and a neighbouring one j is governed by the metric:

$$d = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2} \quad (2)$$

Computing this metric requires evaluating a square root inside the inner loop $N(N - 1)$ times. This operation has a higher cost than the additions and multiplications used for the squared distance. The algorithmic complexity remains $\mathcal{O}(N^2)$.

3.2. Squared distance

The Squared Distance optimization addresses part of the arithmetic cost. Since distance metrics and radii are non-negative real numbers a boid is inside a radius if the following condition is satisfied:

$$d^2 < r^2 \iff (x_j - x_i)^2 + (y_j - y_i)^2 < r^2 \quad (3)$$

This transformation removes the *std::sqrt()* instruction from the neighbour classification loop and reduces the comparison to additions, multiplications and threshold checks. Although the complexity remains quadratic $\mathcal{O}(N^2)$ the inner iteration contains fewer high-latency operations. The test is mathematically equivalent for non-negative distances and radii.

3.3. Bounding box

The Bounding Box optimization introduces a spatial pre-filter before performing the complete squared-distance check. Before executing the mathematical operations required by the boid rules the algorithm establishes an axis-aligned bounding box centered on the agent. The extent of this mathematical square is defined by the interaction radius r along both coordinate axes. A neighbouring boid is rejected before calculating d^2 if it fails the following system of inequalities:

$$\begin{cases} |x_j - x_i| < r \\ |y_j - y_i| < r \end{cases} \quad (4)$$

If a neighbour lies outside this localized bounding box the iteration bypasses the squared-distance and force calculations. The pre-filter also introduces two absolute-value operations and additional conditional control flow for every candidate pair. Its effectiveness therefore depends on whether the rejected distance calculations compensate for this extra overhead.

3.4. Grid

The Grid optimization represents a structural overhaul of the neighbour search. The previous approaches scale quadratically because the simulation has no spatial index and each boid scans

the complete flock. The grid discretizes the continuous 2D space into uniform cells whose side length equals the maximum visual interaction radius ($r_{\max}$).

At the start of each frame every boid is inserted into the cell associated with its position:

$$\text{Cell}_X = \lfloor \frac{x}{r_{\max}} \rfloor, \quad \text{Cell}_Y = \lfloor \frac{y}{r_{\max}} \rfloor \quad (5)$$

The grid construction is intentionally performed sequentially before entering the OpenMP parallel region. Parallelizing this stage would require a synchronization strategy to avoid concurrent writes to the same cell, the current sequential construction therefore contributes to the non-parallel fraction of the Grid profile.

When an OpenMP thread updates an agent the search is confined to its current cell and the 8 adjacent cells. The cost of rebuilding the index is $\mathcal{O}(N)$ while the neighbour search phase can be expressed as $\mathcal{O}(N + K)$, K is the number of candidate pairs contained in the inspected cells. If cell occupancy remains bounded this behaviour approaches linear scaling but the worst case remains $\mathcal{O}(N^2)$ when many boids occupy the same cells. Nevertheless, it evaluates far fewer candidate pairs than the global scan.

The sequential grid construction's relative weight increases when the parallel neighbour search phase becomes short.

4. Setup

4.1. Hardware

The underlying hardware architecture features an AMD Ryzen 5 3600X processor based on the Zen 2 microarchitecture. The CPU consists of 6 physical cores and 12 logical threads enabled via Simultaneous Multithreading (SMT) operating at a base clock frequency of 3.8 GHz with a maximum boost clock of 4.4 GHz. The memory subsystem comprises 16 GB of DDR4 RAM operating in a dual-channel configuration.

To prevent the benchmark from being interrupted by automatic system sleep the program invokes the Windows API function *SetThreadExecutionState* using the flags *ES_CONTINUOUS*

and `ES_SYSTEM_REQUIRED`. These flags keep the system awake during execution but they do not lock the processor frequency or eliminate unrelated operating system activity.

4.2. Build

The project is built using CMake (version 3.20 or higher) targeting the C++17 standard. All benchmarks are compiled in Release mode. GNU and Clang use `-O3 -march=native` with OpenMP support, while Microsoft Visual C++ uses `/O2 /Oi /Ot` and `/openmp`. These optimization levels allow compiler auto-vectorization where applicable.

4.3. Benchmark

The benchmark is structured to test various configurations and parameters. For each of them various performance metrics are evaluated. The test parameters are defined as follows:

- *Optimization levels*: 4 distinct algorithmic implementations (*Naive*, *Squared Distance*, *Bounding Box*, *Grid*).
- *OpenMP scheduling strategies*: 2 distinct loop scheduling routines (*Static* and *Dynamic*).
- *Chunk sizes (k)*: 6 scheduling granularities: $k \in \{1, 8, 32, 64, 128, 256\}$.
- *Boid populations (N)*: 4 cases evaluating low, medium and high density scenarios: $N \in \{2000, 5000, 10000, 20000\}$.
- *Thread counts (p)*: 8 thread configurations scaling up to and beyond the physical and logical core limits: $p \in \{1, 2, 4, 6, 8, 12, 18, 24\}$. The inclusion of 18 and 24 threads allows for the analysis of over-subscription beyond the 12 logical execution contexts of the processor.

Each combination of parameters is tested and evaluated. To reduce the influence of transient OS jitter and background processes every individual configuration is subjected to $R = 5$ independent

runs. Within each run the simulation advances through a fixed number of $F = 50$ consecutive frames. The complete pipeline evaluates 1536 parallel and 32 sequential configurations. This produces 1568 configurations and 7840 total simulations.

4.4. Metrics

The performance analysis is done by calculating 5 metrics using the timer provided by `omp_get_wtime()`.

- *Average, minimum and maximum execution time*: For a given test configuration let T_k represent the total wall-clock time elapsed during the $F = 50$ frame loop of the k -th run. The average execution time (T_{avg}) is computed as:

$$T_{\text{avg}} = \frac{1}{R} \sum_{k=1}^R T_k \quad (6)$$

The minimum and maximum runtimes are:

$$T_{\text{min}} = \min(T_1, T_2, \dots, T_R) \quad (7)$$

$$T_{\text{max}} = \max(T_1, T_2, \dots, T_R) \quad (8)$$

- *Parallel speedup*: The Speedup (S) measures the performance gain of the OpenMP execution relative to the sequential implementation of the same profile:

$$S(p) = \frac{T_{\text{seq}}}{T_{\text{par}}(p)} \quad (9)$$

- *Normalized cost per boid*: The time per boid (τ) is the average wall-clock time per boid for one simulation frame:

$$\tau = \frac{T_{\text{avg}} \times 10^6}{\text{Frames} \times \text{Boids}} \quad (10)$$

All calculated metrics are displayed on the console and appended to the `benchmark_results.csv` file. The sequential baseline is included in the same row.

4.5. Measurement scope

The timer surrounds the 50-frame simulation loop. Flock initialization is performed before timing each run and rendering is not part of the benchmark. The measured value therefore describes the update and commit work without SFML drawing cost.

The same deterministic seed is used when the flock is initialized.

4.6. GUI

The real-time graphical visualization is implemented using SFML (Simple and Fast Multimedia Library).

To prevent the graphics subsystem from starving the CPU or causing unbounded execution speeds on high-refresh-rate monitors the rendering loop is capped to a stable 60 frames per second via `setFramerateLimit(60)`.

5. Results analysis

5.1. Sequential baseline

The sequential function is measured separately for every profile and population. It provides the reference for the speedup calculation.

Table 1 shows the algorithmic effect before OpenMP is introduced, grid is already faster than the all-pairs profiles because it reduces the number of candidate neighbours. Bounding Box has the highest runtime while Squared Distance remains close to Naive.

Table 1. Sequential baseline times (T_{seq} in s).

Legend: Values average the sequential references recorded for the static and dynamic benchmark groups.

Profile	$N = 2000$	$N = 5000$	$N = 10000$	$N = 20000$
Naive	0.6405	3.9179	15.5622	61.9215
Sq.D.	0.6646	4.1007	16.2758	65.0527
B.B.	0.9792	6.0633	23.9305	92.0215
Grid	0.1278	0.6730	2.5229	9.8205

The three all-pairs profiles increase rapidly with population while grid follows a lower trajectory at every tested size. At $N = 20000$ it completes the same 50-frame workload in 9.8205 s while sequential Naive requires 61.9215 s.

5.2. Thread scaling

Table 2 compares every thread count at $N = 10000$. Dynamic scheduling and chunk size $k = 64$ are fixed.

Table 2. Thread scaling at $N = 10000$ with dynamic scheduling and $k = 64$.

Legend: Times are means in s; S is the speedup against the corresponding sequential profile.

Profile	1 Th	2 Th	4 Th	6 Th	8 Th	12 Th	18 Th	24 Th
Naive T_{avg}	17.9794	9.2122	4.7714	3.4006	2.7920	2.0896	2.1144	2.1180
Naive S	0.87×	1.69×	3.26×	4.58×	5.57×	7.45×	7.36×	7.35×
Sq.D. T_{avg}	17.6412	8.9810	4.6268	3.2850	2.6494	1.9490	1.9594	1.9712
Sq.D. S	0.92×	1.81×	3.52×	4.96×	6.15×	8.36×	8.32×	8.27×
B.B. T_{avg}	23.7074	12.0432	6.1890	4.3110	3.4172	2.4388	2.5000	2.5412
B.B. S	1.01×	1.98×	3.86×	5.54×	6.99×	9.79×	9.55×	9.40×
Grid T_{avg}	2.5486	1.3240	0.6942	0.4868	0.3962	0.3006	0.3068	0.3124
Grid S	0.99×	1.90×	3.62×	5.17×	6.35×	8.37×	8.20×	8.06×

One OpenMP thread is slower than the sequential path for Naive, Squared Distance and Grid because it retains the parallel-region and synchronization overhead. Bounding Box remains close to the sequential result. Performance improves strongly from two to six threads and continues to improve up to the 12 logical execution contexts.

The values at 18 and 24 threads remain close to the 12 thread result, these configurations oversubscribe the processor and do not provide stable additional performance. Grid keeps the lowest absolute runtime while Bounding Box reports the highest same-profile speedup.

5.3. Parallel efficiency

Parallel efficiency is evaluated for both the thread count p and the chunk size k :

$$E(p, k) = \frac{T_{\text{seq}}}{pT(p, k)}. \quad (11)$$

An efficiency of 100% means that doubling the number of threads halves the runtime. Lower values indicate that synchronization, scheduling and shared-memory traffic consume a larger part of the execution time.

Table 3 reports the measured efficiency at $N = 10000$ with dynamic scheduling and $k = 64$. It includes every tested thread count.

Efficiency decreases gradually from one to 12 threads. The runtime still improves because the

Table 3. Parallel efficiency at $N = 10000$ with dynamic scheduling and $k = 64$.

Legend: $E(p) = S(p)/p$; values are percentages.

Threads	Naive	Sq.D.	B.B.	Grid
1	86.5%	92.4%	100.7%	98.7%
2	84.4%	90.7%	99.2%	95.0%
4	81.5%	88.1%	96.5%	90.6%
6	76.3%	82.7%	92.3%	86.2%
8	69.7%	76.9%	87.4%	79.4%
12	62.0%	69.7%	81.6%	69.8%
18	40.9%	46.2%	53.1%	45.6%
24	30.6%	34.5%	39.2%	33.6%

additional threads compensate for the lower efficiency, bounding Box retains the highest percentage at every thread count because its larger workload provides more work for each parallel region.

The processor has 12 logical execution contexts so the 18 and 24 thread configurations oversubscribe the hardware. Their speedup remains close to the 12-thread result while efficiency falls to at most 53.1% at 18 threads and 39.2% at 24 threads.

5.4. Amdahl interpretation

The benchmark is not a pure Amdahl experiment because memory behaviour and scheduling also change with the thread count. However the model still explains the plateau after 12 threads, grid contains a sequential construction stage and every profile also contains barriers and the final commit phase.

These parts become more visible after the neighbour loop is accelerated because additional software threads cannot remove them and oversubscription adds further scheduling and contention.

5.5. Profile comparison

The four profiles perform different amounts of work before and after OpenMP distributes the outer loop. Table 4 compares their 12-thread dynamic runtimes with $k = 64$. Each value is divided by the Grid runtime at the same population.

Squared Distance removes the square root but retains the complete all-pairs traversal. It becomes slightly faster than Naive after parallelization but the difference remains small when compared with Grid.

Bounding Box adds axis checks before the

Table 4. Runtime relative to Grid at 12 threads with dynamic scheduling and $k = 64$.

Legend: $R = T_{\text{profile}}/T_{\text{Grid}}$; Grid is the reference at $1.00\times$.

Profile	$N = 2000$	$N = 5000$	$N = 10000$	$N = 20000$
Naive	$3.26\times$	$5.78\times$	$6.95\times$	$7.09\times$
Sq.D.	$3.05\times$	$5.35\times$	$6.48\times$	$6.67\times$
B.B.	$3.84\times$	$6.61\times$	$8.11\times$	$8.61\times$
Grid	$1.00\times$	$1.00\times$	$1.00\times$	$1.00\times$

squared-distance test. The rejection step does not recover the cost of the additional absolute-value operations and branches, it remains the slowest profile at every population.

Grid reduces the candidate set before force calculation. Its advantage increases with population as at $N = 20000$ Naive requires $7.09\times$ the Grid time while Bounding Box requires $8.61\times$ the Grid time.

5.6. Population scaling

Table 5 fixes the parallel configuration at 12 threads, dynamic scheduling and $k = 64$ while the population is the only changing parameter.

Table 5. Population scaling at 12 threads with dynamic scheduling and $k = 64$.

Legend: Values are mean runtimes in s for 50 frames.

Profile	$N = 2000$	$N = 5000$	$N = 10000$	$N = 20000$
Naive	0.1096	0.5796	2.0896	8.2530
Sq.D.	0.1024	0.5358	1.9490	7.7718
B.B.	0.1290	0.6624	2.4388	10.0274
Grid	0.0336	0.1002	0.3006	1.1644

The all-pairs profiles retain quadratic growth because every boid scans a larger population. Grid also becomes slower as the fixed simulation area becomes denser but its increase remains much smaller.

At $N = 20000$ Grid requires 1.1644 s while Naive requires 8.2530 s with the same threads, scheduler and chunks. Compared with the sequential Naive this Grid and OpenMP configuration is about $53\times$ faster.

5.7. Scheduling

Table 6 compares static and dynamic scheduling at $N = 5000$. Chunk size remains fixed at $k = 64$ so the difference comes from the scheduling policy.

Table 6. Static and dynamic scheduling at $N = 5000$ with $k = 64$.
Legend: Values are mean runtimes in s.

Profile / Scheduler	2 Th	6 Th	12 Th	18 Th	24 Th
<i>Naive</i> (Static)	2.3280	0.9072	0.5764	0.6886	0.6298
<i>Naive</i> (Dynamic)	2.3230	0.8594	0.5796	0.5850	0.5736
<i>Sq.D.</i> (Static)	2.2810	0.8688	0.5326	0.6466	0.5870
<i>Sq.D.</i> (Dynamic)	2.2766	0.8384	0.5358	0.5286	0.5334
<i>B.B.</i> (Static)	3.0898	1.1378	0.6832	0.8234	0.7514
<i>B.B.</i> (Dynamic)	3.0996	1.1124	0.6624	0.6668	0.6706
<i>Grid</i> (Static)	0.3754	0.1438	0.1054	0.1260	0.1214
<i>Grid</i> (Dynamic)	0.3704	0.1388	0.1002	0.1052	0.1106

Dynamic scheduling is faster in 79.6% of the matched configurations, while its median improvement is 2.2% its advantage remains smaller than the difference among neighbour-search profiles. Grid benefits from dynamic assignment because boids in dense cells can require more neighbour work than boids in sparse cells.

Static scheduling remains competitive when iteration costs are similar. At low thread counts each worker receives many iterations and local differences tend to average out. Near the hardware limit a slow chunk can delay the barrier and dynamic scheduling can reduce this tail.

5.8. Chunk sizes

Chunk sizes are compared through execution time. The profile, scheduler, population and thread count remain fixed while only k changes. The relative runtime of each chunk is

$$R_k = \frac{T_{\text{avg}}(k)}{\min_k T_{\text{avg}}(k)}. \quad (12)$$

The fastest chunk has $R_k = 1$. A value of 1.10 means that the chunk is ten percent slower than the best value for the same profile, scheduler, population and thread count.

Table 7. Chunk size comparison for grid at $N = 10000$, 12 threads and dynamic scheduling.

Legend: Times are means in s; Δ_{64} is the runtime change relative to $k = 64$.

Chunk	T_{avg}	R_k	Δ_{64}
$k = 1$	0.3128	1.055	+4.1%
$k = 8$	0.2964	1.000	-1.4%
$k = 32$	0.2986	1.007	-0.7%
$k = 64$	0.3006	1.014	0.0%
$k = 128$	0.3132	1.057	+4.2%
$k = 256$	0.3418	1.153	+13.7%

Small chunks can reduce runtime when the cost of the boid iterations is uneven. Dynamic scheduling can redistribute the remaining work more frequently and reduce the time spent waiting at the final barrier. The same chunks can become slower when the workload is already balanced because the runtime performs more scheduling operations.

Large chunks reduce scheduling overhead and can improve static scheduling when the iterations have similar cost. Instead they can become slower when one thread receives a heavier final chunk and the other threads reach the barrier earlier.

The best chunk is not identical for every workload. The all-pairs profiles perform a similar outer-loop structure but branch behaviour changes with the boid position. Grid can have stronger iteration differences because boids in dense cells inspect more neighbours than boids in sparse cells. Population and thread count also change how much work remains inside each chunk.

For the Grid workload $k = 8$ is the fastest value at 0.2964 s. Chunk sizes 32 and 64 remain within 1.5% of the minimum time while $k = 256$ is 15.3% slower than the best value. Across all 256 cases $k = 8$ is the fastest value in 126 cases.

5.9. Time per boid

Table 8 uses 12 threads, dynamic scheduling and $k = 64$. This metric removes the effects of the numbers of the frames and of the boids. It shows how the work required for one boid changes as the flock becomes larger.

Table 8. Mean time per boid and frame at 12 threads with dynamic scheduling and $k = 64$.

Legend: Values are in μs per boid per frame.

Profile	$N = 2000$	$N = 5000$	$N = 10000$	$N = 20000$
<i>Naive</i>	1.096	2.318	4.179	8.253
<i>Sq.D.</i>	1.024	2.143	3.898	7.772
<i>B.B.</i>	1.290	2.650	4.878	10.027
<i>Grid</i>	0.336	0.401	0.601	1.164

The normalized cost rises strongly with the population. Every boid scans a larger flock and the work per boid approaches linear growth in N .

Grid also becomes more expensive because the

simulation area remains fixed and the cells become denser. At $N = 20000$ Grid requires $1.164 \mu\text{s}$ per boid and frame while Naive requires $8.253 \mu\text{s}$ and Bounding Box requires $10.027 \mu\text{s}$.

5.10. Minimum and maximum results

Parallel values use dynamic scheduling .

Table 9. Most relevant minimum and maximum measured values.
Legend: Parallel extrema use $k = 64$, times are means over 50 frames.

Metric	Minimum	Maximum
Sequential runtime	0.1278 s Grid, $N = 2000$	92.0215 s B.B., $N = 20000$
12-thread runtime	0.0336 s Grid, $N = 2000$	10.0274 s B.B., $N = 20000$
12-thread speedup	$7.45 \times$ Naive, $N = 10000$	$9.79 \times$ B.B., $N = 10000$
12-thread efficiency	62.0% Naive, $N = 10000$	81.6% B.B., $N = 10000$
Cost per boid	$0.336 \mu\text{s}$ Grid, $N = 2000$	$10.027 \mu\text{s}$ B.B., $N = 20000$

The minimum sequential and parallel runtimes both belong to Grid at $N = 2000$ while the maximum runtimes belong to Bounding Box at $N = 20000$. The neighbour search profile therefore has a larger effect on absolute time than same-profile parallel efficiency.

Bounding Box records the highest 12-thread speedup and efficiency at $N = 10000$ but it also records the highest normalized cost at $N = 20000$. Grid has a lower same-profile speedup but produces the lowest absolute runtime.

5.11. Conclusion

The results show that the useful OpenMP scaling extends up to the 12 logical execution contexts supported by the processor. Most of the performance improvement is obtained from the six physical cores while simultaneous multithreading provides a smaller additional gain. The efficiency analysis confirms the same behaviour: efficiency decreases as the thread count grows and drops sharply with 18 and 24 threads because these configurations oversubscribe the processor.

The optimization strategy has a greater effect than scheduling alone. Squared Distance reduces the arithmetic cost of each comparison but retains the quadratic traversal. Bounding Box introduces

an additional spatial pre-filter although its overhead makes it slower for the measured workloads. Grid achieves the largest improvement because it reduces the number of candidate neighbours before distributing the remaining work among the threads.

Dynamic scheduling is faster in 79.6% of the direct comparisons but its impact remains smaller than the effects of the optimization profile and the thread count. Among all tested configurations a chunk size of 8 provides the lowest median runtime but the optimal chunk still depends on the specific workload.

For $N = 20000$ the best configuration is the 12-thread dynamic Grid implementation with $k = 32$. It completes one benchmark execution in 1.1046 s and is approximately $56 \times$ faster than the sequential Naive implementation.

Overall, the results demonstrate that combining the algorithmic optimization of the neighbour search with an appropriate OpenMP configuration is substantially more effective than increasing parallelism without changing the underlying quadratic algorithm.

5.12. Future work

- *Parallel grid construction*: Build thread-local cell lists and merge them before the update phase.
- *Data layout*: Replace the array of objects with separate arrays for positions, velocities and pending states to obtain a structure of arrays
- *Density tests*: Compare fixed-area and fixed-density population scaling.
- *Standard deviation*: Adding the standard deviation to the saved data would allow a better study of the benchmark stability.